# Deep Swarm 容量根因分析 Agent 设计与落地分享 - 摘要

11:24 2026/10/24
上海市, CN
由AI生成，请在使用前核实

## 概览

本次分享系统介绍了小红书内部面向容量场景打造的 RCA Agent“Deep Swarm”的技术架构与工程实践。系统针对大模型数值幻觉、长程调查易偏离与高耗时等挑战，通过可观测上下文压缩、基于 Session Tree 的分支调度编排与假设验证机制实现了高效诊断。项目在 100 多个用例的金标数据集上取得了约 70% 的准确率，平均完整诊断耗时约 3 分多钟。

## 要点

- Deep Swarm 选定排查方向集中且数据基础明确的容量分析作为首个落地场景，首个可用判断耗时约 90 秒，完整诊断平均耗时约 3 分多钟。
- 引入可观测上下文工程，通过变点检测、动态分段及特征抽取将 3 个指标的 token 消耗由 12000 多降低 80% 以上至 2000 多。
- 设计基于 Session Tree 的会话管理与多分支编排机制，解决长对话上下文相互干扰、路径不直观及人工接管成本高的问题。
- 采用假设验证与固定流程 Tool 减少大模型往返，支持通过精简交接和提前终止未完成分支来加速调查。
- 搭建包含 LLM 语义匹配与程序计分的金标评测流程，并在 100 多个用例的数据集上实现约 70% 的回归准确率。

## 详情

- RCA Agent 面临的挑战与容量场景选型
  - 核心痛点与工程挑战: 处理海量时序指标、日志与变更数据时，大模型在数值

处理上易出现胡编数据或证据的幻觉问题。 00:00:24

  - 长程调查任务中若 Agent 误读线索易导致调查偏离既定方向且无法收敛，使得线上排查时效性不可控。 00:00:40
  - 针对线上故障分秒必争的场景，需要构建 Harness 工程设计，并要求 Agent 在不确定时明确表达不知道且支持人工随时协同介入。 00:01:08
- 容量场景选型依据: 相比全场景告警，容量排查方向更集中在流量、性能、依赖和资源四个维度，问题范围明确且具备良好数据基础。 00:03:46
  - 传统容量告警排查涉及影响范围、机房单双可用区、突发流量原因及处置有效性等繁琐分析，亟需 Agent 加速排查以减少人力消耗。 00:03:00
  - 验证容量场景能力后，计划逐步将该套能力复用覆盖至全公司更多故障排查场景。 00:04:14
- Deep Swarm 的整体架构设计
  - 分层架构概览: 系统整体自顶向下分为入口层、会话与编排层、Agent 运行层、领域能力层以及底层运行支撑机制。 00:04:47
    - 入口层支持容量平台告警、Web 诊断台提问、内部 IM 机器人以及第三方 OpenAPI 调用接入。 00:04:47
    - 会话与编排层使用 Session Tree 管理调查过程，通过 Context Builder 构建分支上下文，并用 Multi-layer Orchestrator 负责分支调度与结果交接。 00:05:06
    - Agent Runner 负责模型判断、工具调用与结果反馈，底层领域能力由 CLI 脚本、Skill 排查 SOP 与 Tool 封装组件连接 Metric、Log、Trace 及变更数据。 00:05:32
    - 运行支撑机制提供模型接入、持久化、运行协商以及 Agent 评估保障体系以支持持续迭代。 00:06:18
- 可观测上下文工程与数据压缩
  - 上下文输入优化策略: 上下文质量决定诊断 Agent 的结论上限，将杂乱原始数

据精简为包含时间范围、异常指标及前后对照的有序证据。 00:07:05

- 时序与变更数据压缩: 围绕变点进行动态分段并提取中位数、全局斜率与方差等特征，同时保留变点前后（如前后 5 分钟）原始数值供模型二次核验。 00:10:12
  - 针对 3 个指标的测试将 Token 消耗从 12000 多降至 2000 多，压缩率达 80% 以上。 00:11:02
  - 对同一次发布的多个批次进行分组聚合，提取操作人与服务名等公共字段，仅保留批次起止时间、放量比例及拓扑相对坐标。 00:11:57
- 链路追踪与日志采样: 对成功率、Exception 及耗时指标进行异常分析以锁定异常时间窗口，在窗口内对失败、慢请求及正常 Trace 各采样 5 条并关联日志，模型在必要时可申请补偿采样且受上限预算控制。 00:13:36

- 基于 Session Tree 的调查编排与管理
  - 传统线性会话的局限: 否定某个排查方向后历史信息残留在上下文会干扰后续判断并大幅增加耗时，且排查路径不直观。 00:15:04
    - 人工追加线索接管会导致上下文相互堆叠干扰；若新开会话又无法继承已采集的有效数据，进一步推高耗时。 00:16:10
  - Session Tree 实现方案: 每个 Entry 保存完整历史并通过父指针连接成树，Worker 分支结合共享事实前缀、自身任务指令与工具调用并行推进调查。 00:16:52
    - 主 Agent 通过 Multi-layer Orchestrator 的 spawn 方法派发并行任务，并使用类似 join 的机制等待结果交接。 00:18:24
    - 分支结束后仅向主分支返回包含关键结论、证据引用及未解决问题的精简结果，避免主分支上下文受到无关信息干扰。 00:20:21
- 假设验证与调查加速机制
  - 假设验证工作流: 先采集指标、内存、QPS、限流等容量数据由模型分析生成公共事实，再基于假设约束查询范围派发调查任务。 00:22:26

    - 首批派发的调查任务数量控制在 2 到 5 个，且最多允许派发两层 Worker 以控制分支数量与执行成本。 00:23:42
    - 若部分任务结果已能充分解释异常且无反证出现，主 Agent 可通过类似 LensCancel 机制提前终止其他进行中的调查，被取消分支保留未完成状态以便后续人工指令继续推进。 00:24:29
  - 固定流程 Tool 封装: 将查询服务拓扑、逐个查变更及合并分析等高频固定步骤封装为单一 Tool，减少模型往返交互以加速交付。 00:25:30
- 可视化诊断面板与多渠道交付
  - 交互界面呈现: 诊断完成后提供可视化面板，顶部展示根因结论并支持点击查看对应证据，左下部展示受影响的服务链路拓扑范围。 00:27:18
  - 多渠道协同推送: 通过多分支同时派发生成 JSON 结构化产物，后端解析根因分析概要直接附加在告警消息中，点击可跳转至可视化详情面板。 00:29:10
- Agent 评测流程与金标数据集构建
  - 评测流程与指标: 从真实告警出发通过 LLM Judge 进行根因与证据的逐项语义匹配并计分，核心统计 SEA1（首个候选根因命中率）与 SEA2（前两个候选根因命中率）。 00:30:43
    - 仅修改 Judge 或修正金标时可复用历史诊断结果重新评测；改动 Agent 逻辑时需重新运行完整诊断评估。 00:31:04
    - 证据匹配度用于指导流程改进，不作为是否命中根因的刚性门槛。 00:32:47
  - 金标构建方式: 由 Agent 生成数据集草稿，人工核对结论与事实后触发入库并由后端校验结构；数据集包含存储告警原文与异常现象的 Input，以及存储根因列表与事实依据的 Output。 00:31:30
- 生产 Bad Case 分析与迭代改进
  - 漏判案例分析: 在某 Circle 节点入口总流量不变但节点流量翻倍的案例中，早期扩列操作与滞后的机房切流时间不一致，加之变更信息缺少扩列语义，导致模型漏召回该根因。

00:33:07

- 针对性迭代措施: 增加专属知识模块让模型感知 Circle 扩列类似 ES 扩分片、会改变上游请求结构并推高下游流量的特定机制；完善变更数据以区分扩列操作与实际切流阶段的时间戳。 00:33:51

- 容量场景落地效果与未来展望
  - 落地数字表现: 首个假设验证返回耗时约 90 秒，完整诊断平均耗时约 3 分多钟；在 100 多个用例的金标数据集上回归准确率达到约 70% 左右。 00:34:59
  - 未来演化方向: 现阶段重点关注流量问题诊断，长远目标是实现覆盖风险识别、问题诊断、方案制定到治理执行的容量全流程 AI 托管，其中诊断仍是全流程中最难且最关键的演进核心。 00:36:55
    - 持续沉淀服务特征、依赖拓扑与历史排查经验以建立企业运维诊断知识库，为 Agent 提供长期记忆支持。 00:37:35
- 问答交流环节
  - 假设生成机制: 假设完全由 AI 自主提出，Prompt 中约束各假设必须相互独立且具备调查价值；该思路参考了技术文章《How we build an AI SRE Agent》。 00:38:38
  - 告警触发与削峰策略: 系统支持容量告警事件驱动与人工在页面主动发起；当前容量场景告警量相对可控未做削峰，但主干团队在全公司告警诊断中引入了针对未认领、低价值频繁告警的特征识别与过滤削峰机制。 00:39:12
  - 会话树框架落地: 团队在开源框架早期仅有接口定义与设计思路时提前介入并落地实现了 Session Tree 机制。 00:40:37

## 后续行动

**继续参加当天下午后续的技术主题分享活动**

截止日期: 今天下午